



UNIVERSIDAD DE CONCEPCIÓN

Facultad de Ingeniería

Departamento de Ingeniería Informática y Ciencias de la Computación

Carrera: Ingeniería Civil Informática

Tarea 1

Asignatura: Inteligencia Artificial

Nombre:

Benjamín Poblete Castillo

Septiembre de 2026

Índice

1. Modelos	1
1.1. Búsqueda de Costo Uniforme	1
1.2. Búsqueda en Anchura	1
1.3. A*	2
1.4. Búsqueda Voraz del Mejor Primero	3
1.5. Algoritmo Genético	3
2. Propagación de Fuego	4
3. Métricas	4
3.1. Métricas de Supervivencia	4
3.2. Métricas de Tiempo (medido en cantidad de movimientos)	4
4. Resultados	6
5. Conclusiones	7

1. Modelos

Para esta tarea de evaluación de algoritmos se seleccionaron cinco métodos. Entre ellos, se escogieron dos de búsqueda no informada: búsqueda de costo uniforme y búsqueda en anchura; dos de búsqueda informada (o guiada): A* y búsqueda voraz del mejor primero; y, por último, se evaluó un algoritmo genético como enfoque bioinspirado. La aplicación en los agentes consistió en que, al detectarse la propagación de fuego, cada agente recalcula su ruta con el algoritmo correspondiente.

1.1. Búsqueda de Costo Uniforme

Primero, el algoritmo inicia añadiendo el punto de inicio a una cola de prioridad donde se almacenan todos los posibles nodos a visitar. La finalidad de la cola de prioridad es facilitar el momento de sacar el nodo con menor costo. Después, entra al bucle principal del funcionamiento del algoritmo, el cual realiza los siguientes pasos:

- Extrae el nodo con el menor costo para moverse.
- Revisa si este nodo es el final del camino; si es así, retorna todo el camino recorrido más este nodo; si no, continúa con el bucle.
- Explora todos los vecinos alcanzables desde la posición inicial y estos son agregados a la cola de prioridad. Aquí se hacen dos validaciones: que el nodo no haya sido visitado, o que, si lo fue y el costo del nuevo camino es menor, se actualice.

El **while** es el corazón de este algoritmo, ya que mediante sus iteraciones se encuentra el camino a la solución. No obstante, si la ejecución escapa del bucle sin retornar nada, esto significa que el algoritmo no encontró un camino a la solución; por esto, la función retorna *None, None*.

1.2. Búsqueda en Anchura

El funcionamiento del algoritmo **BFS** dentro del código comienza revisando si el nodo inicial es exactamente el nodo de término, para evitar entrar al bucle sin necesidad de hacerlo. Luego de esta validación, se define una cola, que es la encargada de que se recorran todos los nodos que entran (gracias a *FIFO* se logra que el algoritmo sea más eficiente en la práctica, y es lo que garantiza la exploración de este algoritmo). Posterior a esto, comienza el **while** de funcionamiento del algoritmo, el cual tiene como condición seguir ejecutándose mientras tenga elementos dentro de la *queue*.

Bucle:

- Primero se obtiene el primer nodo de la cola; con base en este nodo se trabaja durante la iteración.
- Si el nodo actual es el objetivo, realiza un **break** para salir del bucle y se marca **destino_encontrado** como **True**.

- Si no se detuvo, el **while** revisa todos los vecinos del nodo actual: si alguno de estos fue visitado se ignora; de lo contrario, se agrega a la cola y se marca como visitado.
- Si uno de los vecinos es el nodo final, se termina el bucle principal.

Cuando termina la ejecución del bucle, se revisa el resultado: si el algoritmo no marcó que encontró un camino, retorna *None, None*; pero si se encontró, a partir del nodo final se reconstruye la ruta hasta el nodo inicial, garantizando la ruta óptima.

1.3. A*

El algoritmo A* comienza verificando que el nodo inicial sea distinto del final para evitar entrar en el bucle sin razón. Luego, define dos listas: la primera almacena todas las celdas no visitadas, y la segunda almacena las celdas ya visitadas y sus detalles (quién es el padre, el costo y la heurística); de esta forma, se parte teniendo la información de todo el mapa. Posteriormente, se crea una cola de prioridad (que garantiza extraer siempre el nodo más cercano o más fácil de alcanzar) para almacenar los posibles movimientos; es ahí donde parte añadiendo la posición de inicio, tras lo cual se está listo para ingresar al bucle **while** (cuya condición de término es que la cola esté vacía):

- Se realiza un **pop** para extraer la casilla con la que se trabajará. Si es una que ya fue revisada anteriormente, se salta; de lo contrario, se marca como visitada.
- Se verifica si la celda actual es la final para salir del bucle y marcar el camino como encontrado; si no, continúa de forma normal.
- Se expanden todos los vecinos del nodo actual. Si no fue visitado, se calculan los nuevos costos de g^1 , h^2 y f^3 para estos vecinos; si ya ha sido visitado y resulta en un camino menor, se actualizan en la lista de detalles.

Al salir del bucle, se verifica si el algoritmo marcó el camino como encontrado; si no se encuentra salida, se retorna *None, None*. Si el bucle marcó el camino como encontrado, se realiza una reconstrucción del camino recorrido, similar a la de BFS, iniciando desde el final hacia atrás.

La función heurística utilizada fue la distancia de Manhattan.

¹ g : costo para llegar hasta ese nodo

² h : evaluación en la función heurística

³ f : suma de $g + h$

1.4. Búsqueda Voraz del Mejor Primero

El algoritmo de búsqueda voraz del mejor primero (*Greedy Best-First*) comienza verificando si la posición de inicio es igual a la de término; si es así, retorna inmediatamente un costo de 0 y la misma casilla, evitando entrar al bucle sin necesidad. Tras esta comprobación, se inicializa una matriz booleana para el registro de las celdas ya visitadas y se crea una cola de prioridad. En esta cola se guarda la posición de inicio junto con el valor de la función heurística, el costo acumulado, el nodo actual y el camino recorrido (como formato de guardado). Gracias a que la heurística está en la primera posición de la tupla del nodo, la cola de prioridad ordena automáticamente los elementos, garantizando siempre extraer primero la casilla más cercana según la función heurística. Luego de esto, se ingresa al bucle **while**, el cual se mantiene iterando mientras la cola no esté vacía:

- Se realiza un **pop** para extraer la casilla más prometedora (con menor valor de h).
- Se verifica si el nodo actual corresponde a la meta; de ser así, se termina la búsqueda retornando el costo acumulado y el camino recorrido, en conjunto con el nodo.
- Si aún no se llega al fin, se revisa si la celda ya fue visitada: si no lo ha sido, se marca como visitada en la matriz y se exploran los vecinos transitables.
- Para cada vecino no visitado, se calcula su distancia Manhattan hacia la salida más el costo del movimiento, se extiende la lista del camino incluyendo al vecino y se inserta la información de la nueva casilla en la cola de prioridad.

Al salir del bucle, si la cola se vacía sin haber alcanzado el final, se retorna *None, None*.

1.5. Algoritmo Genético

El algoritmo genético modela la búsqueda de rutas representando cada camino como un individuo, el cual está definido por una secuencia de genes que corresponden a desplazamientos ortogonales (dx, dy) (o discretos, como indica la guía). Para iniciar, se genera una población de 25 individuos aplicando una inclinación direccional: con un 65 % de probabilidad los movimientos se orientan hacia la casilla donde está la salida y un 35 % son seleccionados de forma completamente aleatoria (con la finalidad de facilitar la exploración y evitar obstáculos). Tras definir esta población inicial, comienza la evolución, que se ejecuta durante un máximo de 25 generaciones. Cada generación realiza lo siguiente:

- Se evalúa el *fitness* de cada individuo simulando su posible recorrido sobre la grilla. Si un individuo alcanza la salida, recibe un puntaje muy alto y una bonificación por longitud; si no logra llegar a la meta, su puntuación se calcula según su cercanía Manhattan a la meta y el avance logrado desde el inicio, pero se aplican penalizaciones si la ruta forma bucles redundantes o atraviesa zonas de gran congestión.
- Si en cualquier momento un individuo llega a la celda final, se interrumpe la evolución y se retorna su camino como la solución encontrada.

- Se aplica elitismo para seleccionar al mejor individuo de la generación, copiándolo directamente a la siguiente población sin alteraciones para asegurar que no se pierda el mejor candidato a salida.
- Para generar la nueva población de la siguiente generación, se seleccionan los padres por método de torneos de tres individuos al azar (quien gana es el de mayor *fitness*), se combinan mediante cruce en un punto y se aplica una mutación con un 15 % de probabilidad de modificar pasos aleatorios para aumentar la variabilidad.

Al salir del ciclo de generaciones, si ningún individuo logró completar el camino hasta la salida, el algoritmo implementa *sliding horizon*, retornando la trayectoria del individuo que logró acercarse más a la salida.

2. Propagación de Fuego

En el método de propagación se seleccionan aleatoriamente los siguientes parámetros:

- **Tiempo de espera de propagación:** Se selecciona de forma aleatoria entre 2 y 3 movimientos antes de la próxima propagación.
- **Vecinos quemados:** Se eligen entre 2 y 4 vecinos que se quemarán.

3. Métricas

Las métricas utilizadas para medir el desempeño de los algoritmos se dividen en dos grupos:

3.1. Métricas de Supervivencia

- **Cantidad promedio:** Cantidad promedio de sobrevivientes.
- **Tasa promedio:** Porcentaje promedio de sobrevivientes después de las iteraciones.
- **Desviación estándar:** Muestra qué tanto varía la cantidad de sobrevivientes entre iteraciones.

3.2. Métricas de Tiempo (medido en cantidad de movimientos)

- **Promedio:** Cantidad promedio de movimientos hasta alcanzar la salida.
- **Desviación estándar:** Qué tanto varía la cantidad de movimientos que tardan los agentes en encontrar la salida entre iteraciones.
- **Valores mínimo y máximo:** El agente que más tardó y el que menos tardó en encontrar una salida.

La idea de medir todo mediante promedios radica en que, al realizarse 80 iteraciones por mapa y algoritmo, se genera una cantidad masiva de datos. Por lo tanto, mediante los promedios esto se puede simplificar, mientras que la desviación estándar ayuda a saber qué tan representativos son dichos promedios respecto a los datos generados.

4. Resultados

Tabla con los resultados obtenidos tras la ejecución; las pruebas se realizaron mediante 80 iteraciones con 80 agentes en mapas de 50×50 .

Cuadro 1: Tasa porcentual de supervivencia promedio por mapa (80 repeticiones).

Algoritmo	Mapa 1 (Alta densidad) Tasa $\pm$ DE	Mapa 2 (Densidad media) Tasa $\pm$ DE	Mapa 3 (Baja densidad) Tasa $\pm$ DE
Costo Uniforme (UCS)	41.38 % $\pm$ 10.25 %	57.11 % $\pm$ 10.66 %	58.97 % $\pm$ 9.88 %
Amplitud (BFS)	53.92 % $\pm$ 2.35 %	75.92 % $\pm$ 3.71 %	92.50 % $\pm$ 3.46 %
A*	39.48 % $\pm$ 10.72 %	59.30 % $\pm$ 9.41 %	59.12 % $\pm$ 9.08 %
Greedy Best-First	90.08 % $\pm$ 19.07 %	49.53 % $\pm$ 7.61 %	34.22 % $\pm$ 7.91 %
Algoritmo Genético	0.00 % $\pm$ 0.00 %	32.45 % $\pm$ 7.65 %	33.05 % $\pm$ 7.80 %

Cuadro 2: Cantidad promedio de agentes sobrevivientes (sobre un total de 80 agentes por prueba).

Algoritmo	Mapa 1 (Alta densidad) Sobrevivientes	Mapa 2 (Densidad media) Sobrevivientes	Mapa 3 (Baja densidad) Sobrevivientes
Costo Uniforme (UCS)	33.10	45.69	47.17
Amplitud (BFS)	43.14	60.74	74.00
A*	31.59	47.44	47.30
Greedy Best-First	72.06	39.62	27.38
Algoritmo Genético	0.00	25.96	26.44

Cuadro 3: Distribución promedio del tiempo de despeje medido en turnos de simulación.

Algoritmo	Mapa 1 (Cuello botella) Media $\pm$ DE	Mapa 2 (Laberinto) Media $\pm$ DE	Mapa 3 (Dispersión) Media $\pm$ DE
Costo Uniforme (UCS)	150.97 $\pm$ 9.34	96.54 $\pm$ 8.88	86.85 $\pm$ 4.10
Amplitud (BFS)	112.15 $\pm$ 0.95	81.34 $\pm$ 5.02	72.60 $\pm$ 1.51
A*	149.39 $\pm$ 10.44	93.36 $\pm$ 6.62	84.86 $\pm$ 4.36
Greedy Best-First	120.51 $\pm$ 9.44	78.94 $\pm$ 6.10	77.45 $\pm$ 8.82
Algoritmo Genético	0.00 $\pm$ 0.00	117.99 $\pm$ 7.47	111.79 $\pm$ 6.72

Nota general: A lo largo de las tablas, la sigla **DE** corresponde a la **Desviación Estándar** muestral obtenida sobre las 80 repeticiones de cada escenario.

Cuadro 4: Valores extremos (mínimo y máximo) de turnos registrados para despejar cada escenario.

Algoritmo	Mapa 1		Mapa 2		Mapa 3	
	Mín.	Máx.	Mín.	Máx.	Mín.	Máx.
Costo Uniforme (UCS)	124	179	79	125	78	98
Amplitud (BFS)	110	115	74	93	72	80
A*	113	183	79	108	75	97
Greedy Best-First	100	148	68	101	58	94
Algoritmo Genético	0	0	99	137	99	132

Cuadro 5: Tiempo computacional total acumulado por algoritmo (240 simulaciones en total).

Algoritmo	Tiempo Transcurrido (s)	Formato HH:MM:SS
Greedy Best-First Search	38.55	00:00:38.55
Búsqueda A* (A-Star)	126.65	00:02:06.65
Búsqueda en Amplitud (BFS)	376.29	00:06:16.29
Búsqueda de Costo Uniforme (UCS)	453.40	00:07:33.40
Algoritmo Genético	4269.04	01:11:09.04

5. Conclusiones

Gracias a los resultados obtenidos después de las pruebas, podemos generar múltiples conclusiones sobre los algoritmos a partir de la evaluación de las diferentes métricas. Las conclusiones más importantes que se pueden extraer son:

- En mapas abiertos o con mayor cantidad de casillas disponibles para movimiento (Mapa 2 y Mapa 3), la búsqueda en anchura (BFS) demostró un gran rendimiento, logrando la mayor tasa de supervivencia con 75.92 % y 92.50 %, y manteniendo la mejor estabilidad con desviaciones de 3.71 % y 3.46 %. Esto demuestra su superioridad debido a que garantiza el camino con el menor número de movimientos, terminando antes de que se propague demasiado el fuego.
- La Búsqueda Voraz del Mejor Primero es el método más efectivo con un 90.08 % de supervivencia en el mapa con cuello de botella, mientras que los métodos de Costo Uniforme y A* apenas alcanzaron entre el 39 % y el 41 %. En un escenario con alta congestión y fuego cerrando accesos rápidamente, estos métodos por su rápida convergencia terminan bloqueando el paso por el cuello de botella, colapsando esa salida y generando un bloqueo.
- A pesar de que el algoritmo Greedy tuvo un porcentaje de éxito muy alto en el mapa con cuello de botella (Mapa 1), en las otras dos alternativas su porcentaje de supervivencia cae drásticamente: 49.53 % en el Mapa 2 y un 34.22 % en el Mapa 3. Como este algoritmo se encuentra guiado únicamente por la función heurística de Manhattan, es muy fácil que sea atraído hacia muros o caminos sin salida, entorpeciendo la salida del agente.
- La Búsqueda de Costo Uniforme y A* obtuvieron resultados relativamente similares en gran parte de las métricas, con una leve mejora en A* gracias a la función heurística que guía el camino hacia la salida. Sin embargo, en tiempo de ejecución en segundos, A* es ampliamente superior, ya que aunque los caminos encontrados sean similares, la convergencia gracias a la distancia de Manhattan es mucho más rápida en el algoritmo A* que en la Búsqueda de Costo Uniforme.

- El enfoque bioinspirado fue el de peor rendimiento global: fracasó rotundamente en el Mapa 1 (0.00 % de supervivencia) y no superó el 33 % en los Mapas 2 y 3. Esto se debe a la falta de listas que permitan el retroceso, por lo que después de cada movimiento la única posibilidad de retroceder es encontrar un camino que retorne por ese lugar, dificultando salir de caminos sin salida; además, los individuos quedan atrapados fácilmente por la evaluación en la función Manhattan, resultando en que el fuego consuma las casillas transitables antes de encontrar la salida.
- Los datos muestran que a mayor cantidad de turnos requeridos para evacuar, menor es la tasa de supervivencia. En el Mapa 1 (cuello de botella), los agentes que utilizaron UCS y A* promediaron cerca de 150 turnos de movimiento y registraron supervivencias bajas ($\approx 40\%$), debido a que cada movimiento de agente incrementa de forma acumulativa la propagación de fuego y el riesgo de quedar atrapados.

Declaración sobre el Uso de Inteligencia Artificial Generativa

En el desarrollo de este informe y la resolución de la tarea se utilizó una herramienta de inteligencia artificial generativa con los siguientes fines:

- Generación de los mapas de prueba.
- Selección de las posiciones iniciales de los agentes.
- Apoyo en la documentación del código.
- Explicación y clarificación de conceptos teóricos.
- Adaptación de código repetitivo (la primera función de simulación fue desarrollada manualmente; al requerirse una estructura análoga en el resto de los módulos, se empleó la IA para replicarla y adaptarla a los demás algoritmos).
- Mejora visual en el formato de salida y presentación de los datos.

Referencias

- [1] GeeksforGeeks. *Uniform Cost Search (UCS) in AI*.
Disponible en: <https://www.geeksforgeeks.org/artificial-intelligence/uniform-cost-search-ucs-in-ai/>
- [2] GeeksforGeeks. *Breadth First Search or BFS for a Graph*.
Disponible en: <https://www.geeksforgeeks.org/breadth-first-search-or-bfs-for-a-graph/>
- [3] GeeksforGeeks. *A* Search Algorithm*.
Disponible en: <https://www.geeksforgeeks.org/dsa/a-search-algorithm/>
- [4] GeeksforGeeks. *Greedy Best First Search Algorithm*.
Disponible en: <https://www.geeksforgeeks.org/dsa/greedy-best-first-search-algorithm/>
- [5] DataCamp. *Tutorial de algoritmo genético en Python*.
Disponible en: <https://www.datacamp.com/es/tutorial/genetic-algorithm-python>